



Universidad Francisco
de Paula Santander
Vigilada Mineducación

Programa de
Ingeniería de Sistemas

SIMULACRO PREVIO1

Análisis de Algoritmo

PUNTO 1. Suponiendo que la compilación se realiza con éxito. Dado el siguiente método en Java, calcule su costo algorítmico y determine su complejidad temporal en notación Big-O. Para el análisis, indique cuántas veces se ejecuta cada ciclo y justifique el resultado obtenido.

```
public static void metodo(int n) {  
    int a=0;  
    for (int i = 0; i < n; i++) {  
  
        for (int j = 0; j < n; j++) {  
  
            for (int k = n; k > 0; k = k / 2) {  
                a+=i+k+n;  
                M(n);  
            }  
  
        }  
  
    }  
    System.out. println (a);  
}
```

rta: $n^2 \text{Log}_3(n)$

```
public static void M(int n) {
```

```
    int x = n * n + 5;
```

```
    for (int i = x; i >= n * n; i = i / 2) {  
        System.out.println(i);  
    }  
}
```

PUNTO 2. Suponiendo que la compilación se realiza con éxito y el valor de n es mayor que 0 y $j=1$, calcule el $T(n)$ y BigO del siguiente de **getUFPS**. Especifique la invariante y desarrolle paso a paso el modelo de complejidad que describe el comportamiento del método. Al frente de cada línea especifique el costo de la Operación elemental (OE). Debe ser explícito escribiendo las sumatorias y demás cálculos matemáticos para el $T(n)$ y BigO para considerar el punto válido. Coloque encima de cada instrucción el costo de cada operación elemental.

```
int algunValor(int n, int j) {  
    int c = 0;  
    for (int k = 1; k < (n * j); k *= 3) {  
        int n2 = j + 10;  
        while (n2 < j) {  
            c += k * n * j;  
            n2++;  
        }  
    }  
    return c;  
}
```

rta: $\text{Log}_3^2(n)$

```
int getUFPS(int n, int j) {  
  
    while (n > 1) {  
        algunValor(n, j);  
        n = n / 9;  
    }  
  
    return n;  
}
```

Punto 3. Se tiene una matriz cuadrada A de tamaño $n \times n$, donde $n \geq 1$. Diseñe un algoritmo que permita determinar si la suma de los elementos de la diagonal principal es igual a la suma de los elementos de la diagonal secundaria. El algoritmo debe retornar **true** si ambas sumas son iguales y **false** en caso contrario. El costo de su solución debe ser como máximo $O(n)$.

static boolean sonIgualesD(int A[][])

Tipo	Matriz	Suma Diagonal Principal	Suma Diagonal Secundaria	Resultado
✓ Cumple	$\begin{bmatrix} 2 & 1 & 2 \\ 3 & 4 & 3 \\ 2 & 1 & 2 \end{bmatrix}$	$2 + 4 + 2 = 8$	$2 + 4 + 2 = 8$	✓ Iguales
✗ No cumple	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 8 & 5 \end{bmatrix}$	$1 + 0 + 5 = 6$	$3 + 0 + 7 = 10$	✗ Diferentes

Punto 2. Dados los siguientes dos métodos en Java, ambos diseñados para calcular y retornar la suma de los elementos ubicados en las posiciones pares de un vector de enteros, analice su costo algorítmico. Determine : ¿Cuál es el más eficiente? (Argumente).

```
public static int suma1(int[] v) {  
    int suma = 0;  
  
    for (int i = 0; i < v.length; i++) {  
        if (i % 2 == 0) {  
            suma += v[i];  
        }  
    }  
  
    return suma;  
}
```

```
public static int suma2(int[] v) {  
    int suma = 0;  
  
    for (int i = 0; i < v.length; i = i + 2) {  
        suma += v[i];  
    }  
  
    return suma;  
}
```

Justifique su respuesta a partir del número de iteraciones y de las instrucciones ejecutadas; no es suficiente indicar únicamente la notación Big-O.

Solución Punto 3:

```
static boolean sonIgualesD(int A[][]){  
  
    int n = A.length;  
    int sumaPrincipal = 0;  
    int sumaSecundaria = 0;  
  
    for (int i = 0; i < n; i++) {  
        sumaPrincipal += A[i][i];  
        sumaSecundaria += A[i][n - i - 1];  
    }  
  
    return sumaPrincipal == sumaSecundaria;  
}
```

Costo

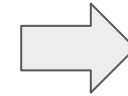
Línea de código	Operación	Tn	
int n = A.length;	Obtener tamaño de la matriz +inicialización	2	
int sumaPrincipal = 0;	Declaración+Asignación	2	
int sumaSecundaria = 0;	Declaración+Asignación	2	
for (int i = 0; i < n; i++)	Declaración+Asignación (int i=0)	2	Se ejecuta n Veces
i < n	Comparación (se ejecuta n+1 veces)	1	
i++	Incremento (se ejecuta n veces)	2	
sumaPrincipal += A[i][i];	Acceso + suma (n veces)+asignación	3	
sumaSecundaria += A[i][n - i - 1];	Acceso + suma (n veces)+ Operación(n-i-1)+asignación	4	
return sumaPrincipal == sumaSecundaria;	Comparación	1	

Costo

$$T(n) = 9 + \sum_{i=0}^{n-1} (1 + 2 + 3 + 4) + 1$$

$$T(n) = 10 + \sum_{i=0}^{n-1} 10 \longrightarrow (n-1) - 0 + 1 = n$$

$$\sum_{i=0}^{n-1} 10 = 10n$$



$O(n)$

$$\sum_{i=a}^b c = c \cdot (b - a + 1)$$

$$T(n) = 10 + 10n$$